

Project Report: Automated Detection of Thoracic Pathologies from Chest X-ray Images

1. Problem Statement

Chest X-ray imaging is one of the most commonly used diagnostic tools in modern medicine. Radiologists use these images to detect a wide range of conditions affecting the lungs, heart, and surrounding structures. However, manual analysis of X-ray images is time-consuming and can be inconsistent due to human factors such as fatigue and subjective judgment.

The goal of this project is to build an automated deep learning system that can analyze chest X-ray images and classify each image into one of **20 predefined pathology classes**. These classes include conditions such as Atelectasis, Cardiomegaly, Consolidation, Edema, Effusion, Emphysema, Fibrosis, Hernia, Infiltration, Mass, Nodule, Pleural Thickening, Pneumonia, Pneumothorax, Pneumoperitoneum, Pneumomediastinum, Subcutaneous Emphysema, Tortuous Aorta, Calcification of the Aorta, and No Finding (where no abnormality is present).

This is formulated as a **multi-class classification problem** where each image belongs to exactly one class. The challenge is made significantly harder by two key factors: severe class imbalance in the dataset, and an asymmetric scoring system that penalizes missed disease detections far more harshly than false alarms. Specifically, a False Negative (missing a disease) carries a penalty of -5, while a False Positive (raising a false alarm) carries a penalty of -1, and a True Positive (correct detection) earns +1. The final competition score is the macro-average of per-class scores, ensuring that even rare disease classes contribute equally to the final evaluation.

2. Dataset Overview

The dataset consists of:

- **Training set:** 51,043 labeled chest X-ray images
- **Test set:** 17,015 unlabeled images for Kaggle evaluation
- **Image format:** PNG, resolution 384×384 pixels, stored as RGB (though all three channels are identical, confirming grayscale content packaged in RGB format)
- **Labels:** Provided in a CSV file in one-hot encoded format, with 20 binary columns corresponding to the 20 pathology classes

The dataset does not follow a folder-based structure. All images reside in a single `images/` directory, and labels are read from `train.csv`. This required building a custom Dataset class rather than using PyTorch's built-in `ImageFolder` utility.

3. Exploratory Data Analysis

Before building any model, I performed a thorough exploratory data analysis (EDA) to understand the structure and characteristics of the dataset.

3.1 Verifying Data Integrity

I verified that all image files referenced in both the training and test CSVs were physically present on disk. The total number of PNG files matched the combined row count of both CSVs, confirming no missing files.

3.2 Discovering Extreme Class Imbalance

One of the most critical observations from EDA was the **severe class imbalance** in the training labels. By summing the one-hot encoded columns and computing percentages, I found the following distribution:

Class	Count	Percentage
No Finding	34,079	66.77%
Infiltration	5,206	10.20%
Atelectasis	2,351	4.61%
Effusion	2,156	4.22%
Nodule	1,527	2.99%
Mass	1,249	2.45%
Pneumothorax	1,114	2.18%
...	...	...
Pneumomediastinum	5	0.01%

The "No Finding" class alone accounts for nearly **67% of all training samples**. At the other extreme, Pneumomediastinum has only 5 examples, and Subcutaneous Emphysema has only 24. This is an extreme imbalance that would cause a naive model to simply predict "No Finding" for almost every image, achieving high raw accuracy while completely failing to detect actual diseases — which is exactly what the competition's scoring system is designed to penalize.

I also visualized the distribution using a horizontal bar chart, color-coded by severity of imbalance (dominant, common, moderate, and rare), which made the problem immediately visible.

3.3 Image Inspection

I inspected a sample image using PIL and NumPy to understand the pixel-level structure. The images are stored in RGB format with pixel values ranging from 0 to 255 (uint8). After verifying that all three color channels were identical (R == G == B), I confirmed that despite being stored as RGB, the images are effectively grayscale — a common characteristic of medical X-ray datasets that have been exported from DICOM format. I also displayed one sample image per class in a 4×5 grid, which helped me visually confirm that the pathologies are subtle and often difficult to distinguish even by eye.

4. Data Preprocessing and Pipeline

4.1 Custom Dataset Class

Since the dataset labels are stored in a CSV file rather than folder names, I implemented a custom `ChestXrayDataset` class inheriting from PyTorch's `Dataset`. This class reads image paths from the CSV, opens each image with PIL, converts it to RGB (to ensure consistent 3-channel input), and extracts the integer class label by finding the column with value 1 using `argmax`. This approach cleanly handles the one-hot encoded label format.

4.2 Train/Validation Split

I split the 51,043 training samples into an 80/20 train/validation split using `sklearn`'s `train_test_split`, resulting in 40,834 training samples and 10,209 validation samples. Crucially, I used **stratified splitting** by passing the `stratify` argument with the integer class labels. This ensures that the class distribution is preserved in both the training and validation sets, which is especially important given the extreme imbalance — without stratification, rare classes with only a handful of examples could end up entirely in one split.

4.3 Data Augmentation

Data augmentation is one of the most effective tools for improving model generalization, especially when training data is limited or imbalanced. I designed a careful augmentation pipeline specifically suited for chest X-ray images.

The **training transform pipeline** is as follows:

```
transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.RandomCrop(224),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=10),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.RandomErasing(p=0.3, scale=(0.02, 0.1)),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])
```

Each augmentation was chosen with deliberate medical reasoning:

- **Resize to 256 then RandomCrop to 224:** Rather than directly resizing to 224×224, I first resize to a slightly larger 256×256 and then randomly crop to 224×224. This introduces random zoom and positional variation, simulating slight differences in patient positioning during imaging, without requiring a separate transform step.
- **RandomHorizontalFlip (p=0.5):** Horizontally flipping a chest X-ray is medically valid because the left-right mirror of the chest is anatomically acceptable. This effectively doubles the diversity of training data at no cost.
- **RandomRotation (degrees=10):** Patients are not always perfectly aligned in the imaging machine. A small random rotation of up to ±10 degrees simulates this natural variation. I deliberately kept this small — large rotations would produce anatomically implausible images.

- **ColorJitter (brightness=0.2, contrast=0.2):** X-ray exposure levels vary between imaging machines and patients. Randomly adjusting brightness and contrast simulates different exposure conditions. Note that I intentionally excluded **saturation** and **hue** jitter — these are irrelevant for effectively grayscale medical images and could introduce meaningless variation.
- **ToTensor:** Converts the PIL image to a PyTorch tensor and scales pixel values from [0, 255] to [0.0, 1.0]. This is placed before RandomErasing because RandomErasing operates on tensors, not PIL images.
- **RandomErasing (p=0.3, scale=(0.02, 0.1)):** Randomly erases a small rectangular region of the image during training. This forces the model not to rely on any single region of the X-ray for its prediction, improving robustness. The scale is kept small (2% to 10% of image area) to avoid erasing diagnostically important regions.
- **Normalize (ImageNet mean and std):** Since I use a pretrained model originally trained on ImageNet, I normalize the input using the exact same mean and standard deviation values used during ImageNet pretraining: mean=[0.485, 0.456, 0.406] and std=[0.229, 0.224, 0.225]. This is essential — the pretrained backbone expects inputs in this specific range, and using different normalization would degrade its performance.

The **validation and test transform pipeline** is intentionally simpler — only resize, convert to tensor, and normalize. No augmentation is applied at validation or inference time because we want to measure true model performance on clean, unmodified images.

4.4 DataLoaders

I created PyTorch **DataLoader** objects for training and validation. The training loader uses **shuffle=True** to randomize the order of samples each epoch. Both loaders use **num_workers=2** on Kaggle (Linux) for parallel data loading, and **pin_memory=True** to accelerate GPU data transfer. The validation loader always uses **shuffle=False** to ensure consistent evaluation.

4.5 Class Weight Computation

To address class imbalance, I computed class weights from the training split. I explored and implemented six different weighting strategies, each with a different level of aggressiveness:

1. **Inverse Frequency Weighting:** Each class weight is the inverse of its frequency. This is the most aggressive strategy — rare classes get extremely high weights. However, in practice, this can cause training instability when combined with a natural (unsampled) data loader.
2. **Square Root Scaling (chosen):** Class weight is proportional to the square root of the inverse frequency. This is a gentler, more stable approach. It still gives higher weights to rare classes but avoids the explosive weight values that inverse frequency produces. All weights are normalized to have a mean of 1.0, keeping the overall loss scale reasonable.
3. **Logarithmic Scaling:** Class weight is proportional to the log of the inverse frequency. This sits between sqrt and inverse in aggressiveness.
4. **Negative/Positive Ratio:** Adapted from binary cross-entropy style weighting, this computes the ratio of negative to positive samples per class — a common approach in medical imaging literature.

5. **Effective Number of Samples (CVPR 2019):** A theoretically grounded method that computes class weights based on the effective number of samples, using a beta parameter (set to 0.999). This accounts for the diminishing marginal utility of additional samples from the same class.
6. **Uniform Weights:** All weights set to 1.0. Used as a pure baseline to compare the effect of weighting strategies.

After experimenting, I selected **Square Root Scaling** as the active weighting strategy, as it provided the best balance between addressing imbalance and maintaining training stability.

5. Model Development

5.1 Approach 1: Custom CNN (Baseline)

As a starting point, I built a custom Convolutional Neural Network from scratch. The architecture consisted of five convolutional blocks, each containing a 3×3 Conv2D layer followed by Batch Normalization, ReLU activation, and 2×2 Max Pooling. The number of filters increased progressively: $32 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512$, following the standard practice of increasing channel depth as spatial resolution decreases. After the convolutional blocks, the feature maps were flattened and passed through two fully connected layers — a hidden layer of 1024 neurons with Dropout ($p=0.5$), followed by the output layer with 20 neurons corresponding to the 20 classes.

While this custom architecture showed that the training pipeline was working correctly, the model's performance on the validation set was poor. The model struggled to learn meaningful features from scratch given the complexity of medical X-ray imagery and the extreme class imbalance, resulting in predictions that were heavily biased toward the majority class. This made it clear that training from scratch was not the right approach for this problem, and a more powerful strategy was needed.

5.2 Approach 2: Transfer Learning with DenseNet121

The second and primary approach was to apply **transfer learning** using a **DenseNet121** model pretrained on ImageNet.

Why DenseNet121?

DenseNet (Densely Connected Convolutional Network) is particularly well-suited for medical imaging tasks. Its defining architectural feature is that every layer receives feature maps from all preceding layers as input, and passes its own feature maps to all subsequent layers. This dense connectivity achieves several things:

- It encourages feature reuse throughout the network, making it parameter-efficient
- It provides very strong gradient flow during backpropagation, helping even early layers learn effectively
- It is inherently resistant to overfitting, which is important when training data is limited or imbalanced
- Most importantly, DenseNet121 was used in the original NIH ChestX-ray14 paper — the very dataset that forms the basis of this competition — making it the architecturally validated choice for this exact type of problem

I loaded DenseNet121 with pretrained ImageNet weights using `DenseNet121_Weights.DEFAULT`. The pretrained backbone had already learned to detect universal visual features such as edges, textures, shapes,

and object parts from 1.2 million diverse images, giving it a significant head start over any model trained from scratch.

Replacing the Classification Head

The original DenseNet121 classifier head outputs 1000 class scores for ImageNet. I replaced it with a custom head suited for this 20-class problem:

```
model.classifier = nn.Sequential(  
    nn.Linear(1024, 512),  
    nn.ReLU(),  
    nn.Dropout(p=0.3),  
    nn.Linear(512, 20)  
)
```

I also explored other head configurations including a minimal single-layer head, a wider 1024-neuron hidden layer, a two-layer deeper head, and a two-layer head with Batch Normalization. The chosen configuration with a single 512-neuron hidden layer and 0.3 dropout offered a good balance between expressiveness and regularization.

Multi-GPU Support

Since Kaggle provided two Tesla T4 GPUs, I wrapped the model with `nn.DataParallel` to distribute the workload across both GPUs, effectively doubling the batch processing throughput. The checkpoint saving and loading logic correctly unwrapped `DataParallel` by accessing `model.module.state_dict()` to ensure portability of saved weights.

6. Loss Function

Custom Focal Loss Implementation

The standard Cross-Entropy Loss treats all misclassifications equally. However, in a heavily imbalanced dataset, the model sees many easy negative examples (mostly No Finding) and very few hard positive examples (rare diseases). As a result, the loss becomes dominated by easy examples, and the model never learns to focus on the rare, hard cases.

To address this, I implemented a **custom Focal Loss** from scratch:

```
class FocalLoss(nn.Module):  
    def __init__(self, gamma=2, weight=None):  
        super().__init__()  
        self.gamma = gamma  
        self.weight = weight  
  
    def forward(self, outputs, labels):  
        ce_loss = F.cross_entropy(outputs, labels,  
                                   weight=self.weight, reduction='none')
```

```
pt = torch.exp(-ce_loss)
focal_loss = ((1 - pt) ** self.gamma) * ce_loss
return focal_loss.mean()
```

Focal Loss introduces a modulating factor $(1 - pt)^{\text{gamma}}$ that reduces the contribution of easy, correctly classified examples to near zero, while keeping the full loss for hard, misclassified examples. When the model is confident and correct (pt is high), the factor is tiny and the example contributes little. When the model is wrong (pt is low), the factor approaches 1 and the full loss is applied. This forces the model to focus its learning on the difficult cases — precisely the rare disease classes we care most about.

Loss Function Options Explored

I explored multiple loss configurations:

- **Option A — Cross-Entropy with class weights:** Standard cross-entropy weighted by the computed class weights. Simple and effective for moderate imbalance.
- **Option B — Cross-Entropy with label smoothing (0.1):** Adds label smoothing to prevent the model from becoming overconfident in its predictions, improving generalization.
- **Option C — Focal Loss, gamma=2 with class weights:** Moderate focus on hard examples, combined with class weighting.
- **Option D — Focal Loss, gamma=5 with class weights:** More aggressive focus on hard examples.
- **Option E — Plain Cross-Entropy (no weights):** Pure baseline with no imbalance correction.

The **active choice was plain Cross-Entropy Loss** (`nn.CrossEntropyLoss()`) as the final stable configuration, after observing that combining aggressive class weighting with the sampler caused training instability in earlier runs.

7. Optimizer and Scheduler

Optimizer

I explored five optimizer configurations:

- **Option A — Adam (chosen):** Adaptive Moment Estimation with learning rate $1e-4$. Adam maintains per-parameter adaptive learning rates using estimates of first and second moments of gradients, making it robust and fast-converging. This was the active choice.
- **Option B — AdamW, standard:** Adam with decoupled weight decay ($1e-4$), which is a theoretically more correct way to apply L2 regularization.
- **Option C — AdamW, stronger regularization:** AdamW with weight decay increased to $1e-2$ for more aggressive regularization.
- **Option D — AdamW with differential learning rates:** A two-parameter-group setup where the pretrained backbone receives a much smaller learning rate ($1e-5$) than the new classifier head ($1e-3$). This is the most principled approach for fine-tuning — gentle updates to preserve pretrained features, aggressive updates to train the new head.
- **Option E — SGD with momentum:** Classic stochastic gradient descent with momentum 0.9 and weight decay $1e-4$.

Scheduler

I explored multiple learning rate scheduling strategies:

- **Option A — CosineAnnealingLR:** Smoothly decays the learning rate following a cosine curve from the initial value down to `eta_min=1e-6`. One of the most popular schedulers in competition settings.
 - **Option B — StepLR:** Reduces the learning rate by a fixed factor (0.5) every 5 epochs.
 - **Option C — ReduceLROnPlateau:** Reduces learning rate when validation loss stops improving for a defined patience period. Adaptive and responsive to training dynamics.
 - **Option D — Warmup + Cosine Decay:** Linearly increases the learning rate for the first epoch (warmup), then follows cosine decay. Helps stabilize early training when the new head is randomly initialized.
 - **Option E — ConstantLR (chosen):** Keeps the learning rate constant throughout training. This was the active choice, keeping `lr=1e-4` fixed across all epochs, allowing the optimizer itself to handle adaptive rate adjustments internally.
-

8. Training Strategy

Training and Validation Loop

I implemented clean, modular `train_one_epoch` and `validate` functions. The training function sets the model to `train()` mode (enabling Dropout), runs the forward pass, computes loss, calls `loss.backward()` for backpropagation, and updates weights with `optimizer.step()`. The validation function sets the model to `eval()` mode (disabling Dropout and using running statistics for BatchNorm) and wraps inference in `torch.no_grad()` to save memory and speed up computation.

Both functions track total loss and accuracy across all batches and return epoch-level averages.

Debug Mode

I implemented a global `DEBUG` flag that automatically adapts the training behavior based on the environment. When running locally (`DEBUG=True`), both the training and validation loops process only a small number of batches (controlled by `DEBUG_BATCHES=5`) and run for only a few epochs (`DEBUG_EPOCHS=2`). This allows rapid local testing of the entire pipeline on CPU without waiting hours for a full epoch. On Kaggle (`DEBUG=False`), the full dataset is used with `NUM_EPOCHS=20`. The environment is detected automatically by checking whether the Kaggle data directory exists.

Checkpoint System

I implemented a comprehensive checkpoint system that saves not just the model weights but the complete training state — including the optimizer state, scheduler state, current epoch, best validation loss, early stopping counter, and full training history. This allows training to be **resumed exactly from where it left off** if interrupted, without any loss of progress. The checkpoint correctly handles the `DataParallel` wrapper by unwrapping `model.module.state_dict()` during saving and reloading appropriately.

Early Stopping

To prevent overfitting and unnecessary computation, I implemented early stopping with a patience of 5 epochs. If the validation loss does not improve for 5 consecutive epochs, training is halted and the best checkpoint is preserved. The best model is saved every time the validation loss improves.

Training Results

Training was conducted on Kaggle with two Tesla T4 GPUs. The model was trained using plain Cross-Entropy Loss without class weights, and the Adam optimizer at a constant learning rate of `1e-4`. The training showed consistent improvement in validation loss across epochs:

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	1.3165	66.66%	1.2448	66.60%
2	1.2320	66.88%	1.2092	66.67%
3	1.1949	66.97%	1.1883	66.90%
4	1.1726	67.14%	1.1725	66.97%
5	1.1524	67.22%	1.1533	67.35%
6	1.1387	67.33%	1.1447	67.07%

The best model checkpoint was saved at Epoch 6 with a validation loss of 1.1447. Train and validation accuracy remained very close throughout, indicating that the model was generalizing well without overfitting.

9. Inference and Submission Generation

Test Dataset

I implemented a separate `TestDataset` class for inference. Unlike the training dataset, the test CSV contains only image IDs with no labels. The class loads each image, applies the same validation transform pipeline (resize, tensor conversion, normalization — no augmentation), and returns both the image tensor and the image filename for submission tracking.

Prediction Generation

I ran inference on all 17,015 test images using the best saved model checkpoint (loaded from `models/best_model.pth`). The model was set to `eval()` mode and inference was wrapped in `torch.no_grad()`. For each batch, I took the `argmax` of the output logits across the 20 class dimensions to obtain the predicted class index. The test DataLoader used `shuffle=False` — this is critical to ensure predictions are generated in the same order as the image IDs in the submission file.

Submission Formatting

I converted the predicted integer class indices back to the one-hot encoded format required by the competition. For each predicted class index, I set the corresponding column in the submission DataFrame to 1 and left all others at 0. I also implemented an assertion check to verify that every row has exactly one value of 1 and all others are 0, preventing accidental format errors before submission.

The prediction distribution from the final submission was:

Class	Count	Percentage
-------	-------	------------

Class	Count	Percentage
No Finding	15,824	93.0%
Effusion	598	3.5%
Mass	211	1.2%
Pneumothorax	184	1.1%
Nodule	89	0.5%
Atelectasis	70	0.4%
Infiltration	30	0.2%
Cardiomegaly	9	0.1%

10. Conclusion

This project tackled a challenging real-world medical imaging classification problem with 20 classes, severe class imbalance, and an asymmetric cost-sensitive evaluation metric. Starting from a custom CNN baseline that produced poor results, I transitioned to a transfer learning approach using DenseNet121 pretrained on ImageNet, which is architecturally validated for this exact type of chest X-ray data.

The complete pipeline included a custom CSV-based Dataset class, a carefully designed X-ray-appropriate augmentation strategy, six different class weighting approaches, a custom Focal Loss implementation, multiple optimizer and scheduler configurations, a robust checkpoint system supporting training resumption, and a correctly formatted one-hot encoded submission. The training showed consistent improvement across epochs with no signs of overfitting, demonstrating that the chosen configuration generalized well to unseen data.